

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнила:

Рудникова Виктория

К3343

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Вам нужно привязать то, что Вы делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализуйте моковое API средствами JSON-сервера и подключите к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Имитация работы с API".

Ход работы

Запустим JSON-сервер

В папке с лабораторной (где уже лежат все файлы с сайтом) создадим и заполним моковыми данными файл **db.json**.

```
1  {
2    "users": [
3      {
4        "id": 1,
5        "name": "Тест Тестов",
6        "email": "testovtest@gmail.com",
7        "password": "123456"
8      },
9      {
10       "id": 2,
11       "name": "Снежана Абдурахманова",
12       "email": "snezhana_hi@gmail.com",
13       "password": "qwerty"
14     },
15     {
16       "id": 3,
17       "name": "Максим Ашкудишкин",
18       "email": "ashk220@gmail.com",
19       "password": "password"
20     }
21   ],
22   "listings": [
23     {
24       "id": 1,
25       "title": "Квартира-студия 🏠 метро Владимирская",
26       "address": "Санкт-Петербург, Центральный район",
27       "price": 45000,
28       "image": "studiya_v_centre.jpeg",
29       "rooms": 1,
30       "area": 24,
31       "floor": "4/6",
32       "type": "квартира"
33     }
34   ]
35 }
```

Я добавила несколько пользователей и все объявления, которые есть у меня на сайте.

JSON-сервер запустился и отображает данные:

```
o vorudnikova@MSK-T7GWWG604V lab2 % npx json-server --watch db.json --port 3001
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3001
Press CTRL-C to stop
Watching db.json...

♡( ͡° ͜ʖ ͡° )

Index:
http://localhost:3001/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3001/users
http://localhost:3001/listings
```

```
localhost:3001/listings

Pretty print ☐

[
  {
    "id": "1",
    "title": "Квартира-студия у метро Владимирская",
    "address": "Санкт-Петербург, Центральный район",
    "price": 45000,
    "image": "studiya_v_centre.jpeg",
    "rooms": 1,
    "area": 24,
    "floor": "4/6",
    "type": "квартира"
  },
  {
    "id": "2",
    "title": "2-к. квартира с видом на реку",
    "address": "Санкт-Петербург, Василеостровский район",
    "price": 70000,
    "image": "s_vidom_na_reku.jpeg",
    "rooms": 2,
    "area": 56,
    "floor": "8/16",
    "type": "квартира"
  },
  {
    "id": "3",
    "title": "Загородный дом с террасой",
    "address": "Ленинградская область, Всеволожский район",
    "price": 90000,
    "image": "zagorodny_dom_2.jpg",
    "rooms": 4,
    "area": 120,
    "floor": "2 этажа",
    "type": "дом"
  }
]
```

```
localhost:3001/users
Pretty print ☐
[
  {
    "id": "1",
    "name": "Тест Тестов",
    "email": "testovtest@gmail.com",
    "password": "123456"
  },
  {
    "id": "2",
    "name": "Снежана Абдурахманова",
    "email": "snezhana_hi@gmail.com",
    "password": "qwerty"
  },
  {
    "id": "3",
    "name": "Максим Ашкудишкин",
    "email": "ashk228@gmail.com",
    "password": "password"
  }
]
```

Далее реализуем запросы по API.

- 1) В файле index.html (на главной) удалим статичные карточки и будем вместо этого получать их через API

Fetch для объявлений:

```
191
192     fetch(API_URL + '/listings')
193       .then(function (res) { return res.ok ? res.json() : Promise.reject(new Error('Ошибка загрузки')); })
194       .then(function (data) {
195         loading.remove();
196         if (data && data.length) {
197           data.forEach(function (item) {
198             row.insertAdjacentHTML('beforeend', renderCard(item));
199           });
200         } else {
201           row.innerHTML = '<div class="col-12 text-center text-muted py-4">Объявлений пока нет.</div>';
202         }
203       })
204       .catch(function () {
205         loading.textContent = 'Не удалось загрузить объявления.';
206         loading.classList.remove('text-muted');
207         loading.classList.add('text-danger');
208       });
209     })();
210   </script>
211 </body>
212 </html>
```

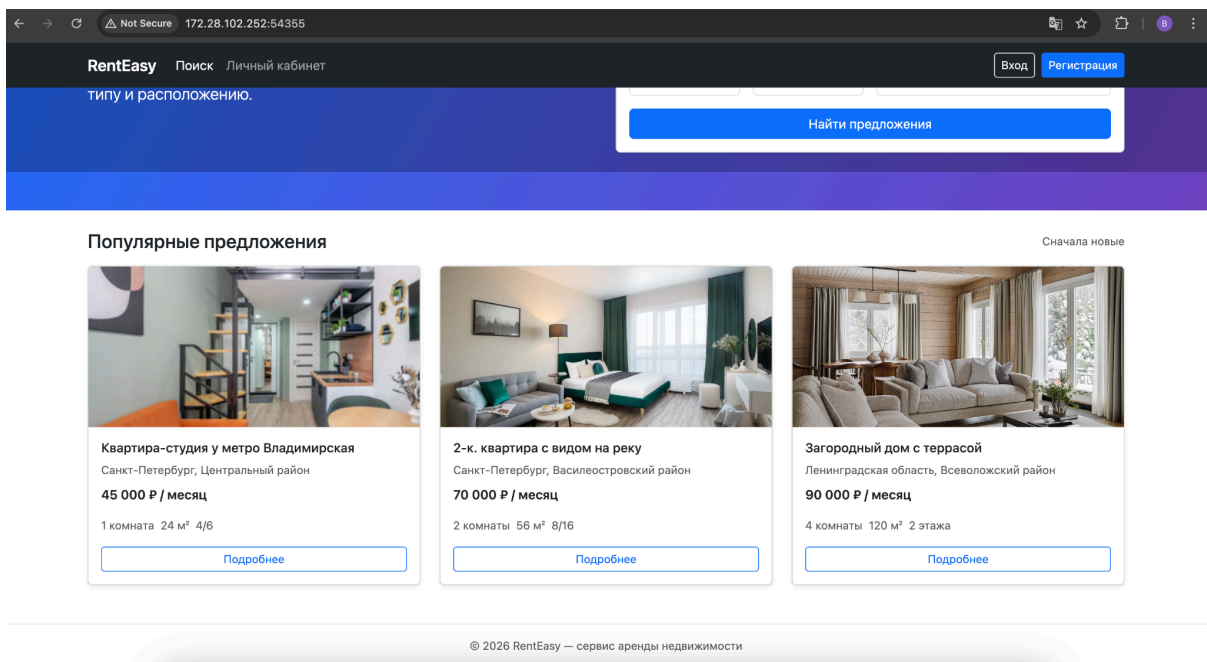
Новая отрисовка карточек:

```

165 (function () {
166   var API_URL = 'http://localhost:3001';
167   var row = document.getElementById('listings-row');
168   var loading = document.getElementById('listings-loading');
169
170   function renderCard(item) {
171     var roomsText = item.rooms === 1 ? '1 комната' : item.rooms + ' комнаты';
172     return (
173       '<div class="col-md-6 col-lg-4">' +
174       '<article class="card h-100 shadow-sm">' +
175         '' +
176         '<div class="card-body d-flex flex-column">' +
177           '<h3 class="h6 card-title">' + (item.title || '') + '</h3>' +
178           '<p class="card-text small text-muted mb-2">' + (item.address || '') + '</p>' +
179           '<p class="fw-semibold mb-3">' + (item.price ? item.price.toLocaleString('ru-RU') + ' Р / месяц' : '') + '</p>' +
180           '<ul class="list-inline small mb-3 text-muted">' +
181             '<li class="list-inline-item">' + roomsText + '</li>' +
182             '<li class="list-inline-item">' + (item.area ? item.area + ' м²' : '') + '</li>' +
183             '<li class="list-inline-item">' + (item.floor || '') + '</li>' +
184           '</ul>' +
185           '<a href="apartment.html?id=' + (item.id || '') + '" class="btn btn-outline-primary btn-sm mt-auto">Подробнее</a>'
186         '</div>' +
187       '</article>' +
188     '</div>'
189   );
190 }
191

```

Страница выглядит точно так же!



Problems	Output	Debug Console	Terminal	Ports
HTTP	2/13/2026 6:47:19 AM	172.28.102.252	Returned 200 in 2 ms	
HTTP	2/13/2026 6:47:19 AM	172.28.102.252	Returned 200 in 3 ms	
HTTP	2/13/2026 6:47:19 AM	172.28.102.252	Returned 200 in 3 ms	
HTTP	2/13/2026 6:47:35 AM	172.28.102.252	GET /profile.html	
HTTP	2/13/2026 6:47:35 AM	172.28.102.252	Returned 301 in 2 ms	
HTTP	2/13/2026 6:47:35 AM	172.28.102.252	GET /profile	
HTTP	2/13/2026 6:47:35 AM	172.28.102.252	Returned 200 in 2 ms	
HTTP	2/13/2026 6:47:35 AM	172.28.102.252	GET /s_vidom_na_reku.jpeg	
HTTP	2/13/2026 6:47:35 AM	172.28.102.252	Returned 304 in 1 ms	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	GET /index.html	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	Returned 301 in 3 ms	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	GET /index	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	Returned 301 in 0 ms	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	GET /	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	Returned 304 in 0 ms	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	GET /zagorodny_dom_2.jpg	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	GET /s_vidom_na_reku.jpeg	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	Returned 304 in 0 ms	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	Returned 304 in 1 ms	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	GET /studiya_v_centre.jpeg	
HTTP	2/13/2026 6:47:37 AM	172.28.102.252	Returned 304 in 0 ms	

Повторим то же самое с пользователями – API позволит проводить валидацию паролей, а также отображать в профиле те данные, которые соответствуют пользователю.

1) Fetch для входа и регистрации

```

297
298 var API_URL = "http://localhost:3000";
299
300 document.getElementById("loginForm").addEventListener("submit", function (e) {
301     e.preventDefault();
302     var email = document.getElementById("loginEmail").value.trim();
303     var password = document.getElementById("loginPassword").value;
304     fetch(API_URL + "/users?email=" + encodeURIComponent(email))
305     .then(function (res) { return res.json(); })
306     .then(function (data) {
307         var user = Array.isArray(data) ? data.find(function (u) { return u.email === email && u.password === password; }) : null;
308         if (user) {
309             window.location.href = "profile.html?name=" + encodeURIComponent(user.name || "") + "&email=" + encodeURIComponent(user.email || "");
310         } else {
311             alert("Неверный email или пароль.");
312         }
313     })
314     .catch(function () { alert("Ошибка соединения с сервером."); });
315 });
316
317 document.getElementById("registerForm").addEventListener("submit", function (e) {
318     e.preventDefault();
319     var name = document.getElementById("registerName").value.trim();
320     var email = document.getElementById("registerEmail").value.trim();
321     var phone = document.getElementById("registerPhone").value.trim();
322     var password = document.getElementById("registerPassword").value;
323     var repeat = document.getElementById("registerPasswordRepeat").value;
324     if (password !== repeat) {
325         alert("Пароли не совпадают.");
326         return;
327     }

```

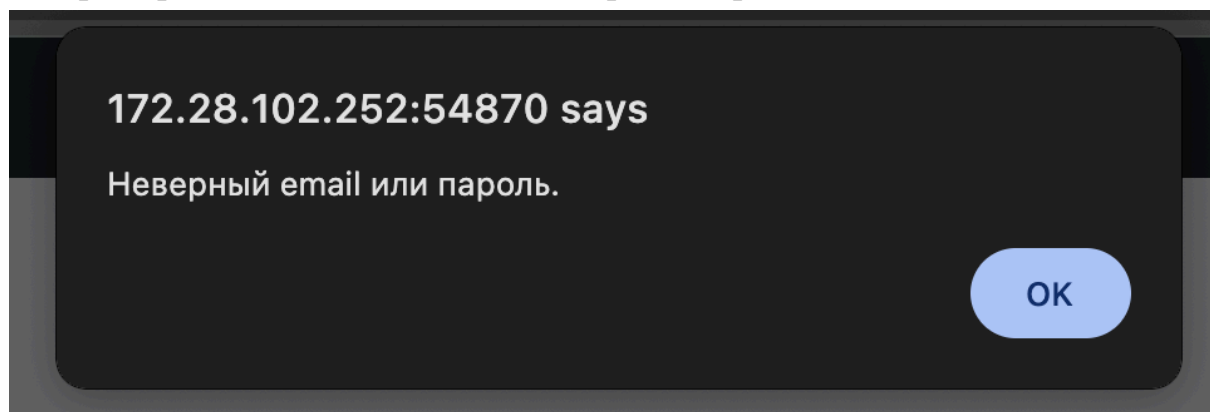
```

328 fetch(API_URL + "/users?email=" + encodeURIComponent(email))
329 .then(function (res) { return res.json(); })
330 .then(function (data) {
331   var exists = Array.isArray(data) && data.length > 0;
332   if (exists) {
333     alert("Пользователь с таким email уже зарегистрирован.");
334     return;
335   }
336   return fetch(API_URL + "/users", {
337     method: "POST",
338     headers: { "Content-Type": "application/json" },
339     body: JSON.stringify({ name: name, email: email, password: password, phone: phone || "" })
340   }).then(function (res) { return res.json(); });
341 })
342 .then(function (user) {
343   if (user && user.id) {
344     window.location.href = "profile.html?name="
345       + encodeURIComponent(user.name || "")
346       + "&email=" + encodeURIComponent(user.email || "");
347   }
348 })
349 .catch(function () { alert("Ошибка соединения с сервером."); });
350 });
351

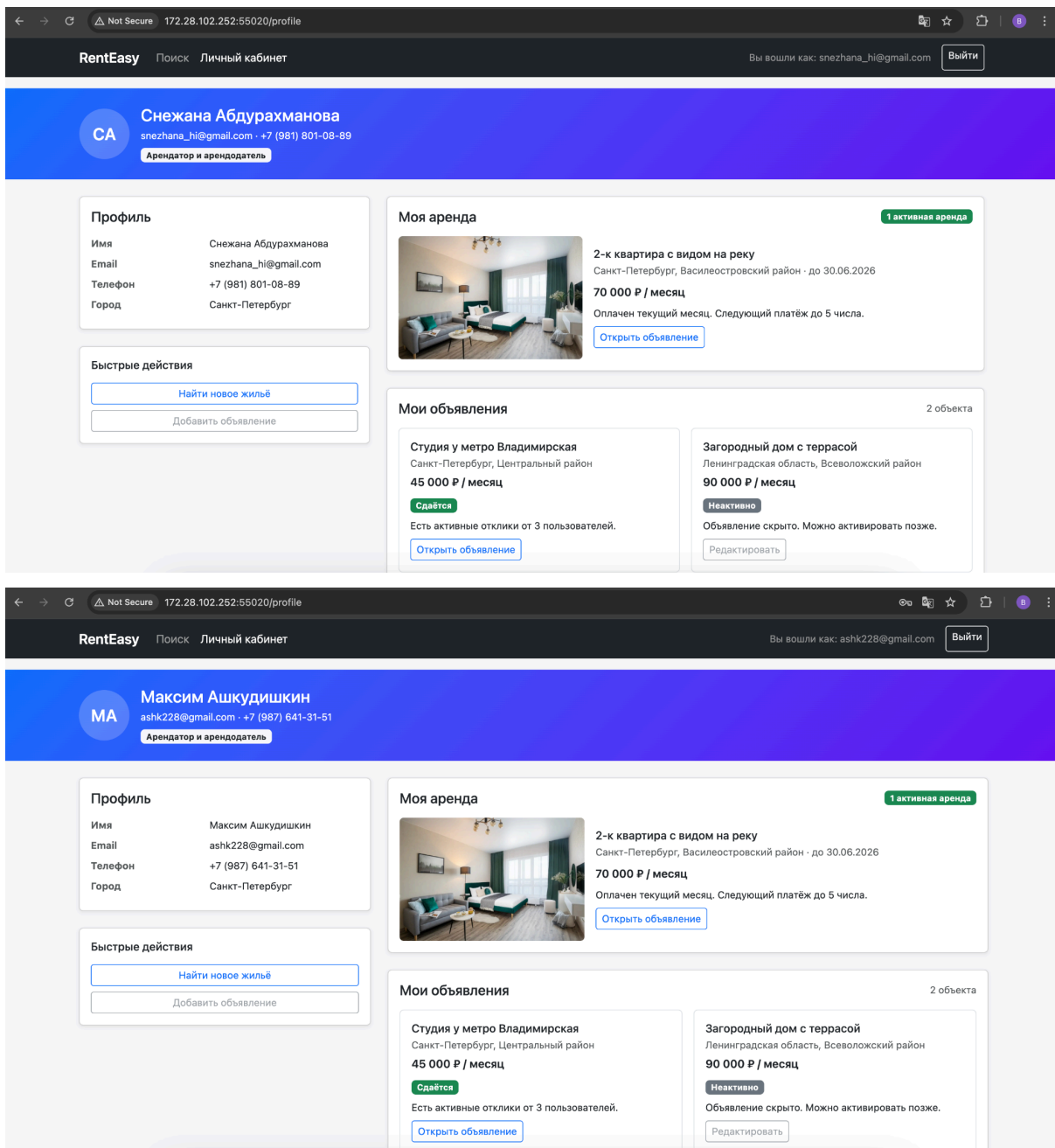
```

2) Fetch для профиля

Теперь страница сообщает нам о неверном пароле



А также в профиле отображается тот, кто входил



Вывод

В лабораторной работе я научилась работать с внешним API и имитировать его с помощью JSON-сервера.